CampusFlow – User Manual

What it is. CampusFlow is a terminal program (C++17) for venue seating, check-in and crowd movement. It runs from one numbered menu and needs no server, database or internet. It uses a hash table (participant lookup), a 2-D seat grid, a segment tree (zone counts, first free seat), a min-heap (waiting list) and a graph (routes, evacuation).

1. Requirements

A C++17 compiler (`g++` 7 or newer) and `make`. Linux/macOS: `sudo apt install g++ make` or Xcode command line tools. Windows: install MSYS2 or MinGW-w64 (add its `bin` folder to PATH).

2. Build and run

Open a terminal in the project folder (the one containing `Makefile`); `include/` holds headers, `Source/` source files, `object/` object files, `tests/` test programs and `samples/` demo inputs.

`make`	builds the program `a.out` (objects go to `object/`)
`./a.out`	starts CampusFlow (in Windows CMD type `a.out`)
`make test`	builds and runs the 5 test suites; ends with "ALL TEST SUITES PASSED"
`make demo`	replays the three sample sessions in `samples/` and compares the output
`make clean`	removes object files, test programs and `a.out`

Without make (Windows CMD) – the three lines below are one command, typed on a single line:

```
g++ -std=c++17 -O2 -Iinclude Source\allocation.cpp Source\hash_table.cpp Source\persistence.cpp
    Source\seat_grid.cpp Source\segment_tree.cpp Source\waitlist_heap.cpp Source\waitlist.cpp
    Source\occupancy.cpp Source\graph_menu.cpp Source\venue_link.cpp Source\main.cpp -o a.out
```

3. Using the program

Type an option number and press Enter. A non-number prints "Please enter a whole number." The venue is a 6×8 grid (rows 0–5, columns 0–7) of 48 seats in 3 zones (0, 1, 2) of 16 seats. Priority is 0–9 (higher is served first).

No.	Option	What it does (prompts)
1	Register participant	Adds a person with no seat (ID, name, priority, preferred zone). Empty or duplicate ID is rejected.
2	Allocate seat	Seat in the preferred zone; overflows to the nearest zone; if the venue is full the person joins the waiting list (ID).
3 / 4	Check in / Mark no-show	Check in: allocated → occupied. No-show: frees the seat and seats the next waiting person.
5 / 6	Reassign / Cancel	Move a person to a free seat (ID, row, column) / remove registration and free the seat.
7	Show participant	State, priority, zone and seat for an ID.
8 / 9	Show seat grid / Zone statistics	Grid legend: F free, A allocated, O occupied, X blocked, R reserved.
10 / 11	Block / Reserve a seat	Takes a free seat out of service / keeps it from automatic allocation (row, column).
12	Load sample data	Registers N people `P101`, `P102`, ... with rotating priority and zone.
13 / 14	Save / Load file	Saves or loads participants and blocked/reserved seats as a text file (file name).
15	Show waitlist	Per zone: higher priority first, then earlier join.
16 / 17	Advance clock / No-show check	Clock moves N minutes; the check works only after 15 minutes and releases seats of people not checked in.
18	Seat all registered	Seats everyone registered without a seat, in ID order.
19	Occupancy report	Capacity, checked-in, free and % per zone: LOW < 30%, NORMAL < 70%, HIGH < 90%, else OVERCROWDED.
20	Simulate arrivals	Checks in everyone still allocated in a chosen zone.
21 / 22	Venue graph / Custom venue	Opens the graph menu (below) / replaces the default venue map with one you type in.
23	Verify consistency	Cross-checks hash table, seat grid and segment tree; should print "Consistent".
0	Exit	Quits the program.

4. Graph menu (option 21)

Default map: Gate, Junction, Zone 0–2, Exit East, Exit West (names are not case-sensitive). **1** display venue; **2** set a location's occupancy; **3** close/open a location; **4** close/open a corridor; **5** set a corridor's congestion likelihood (0–100%); **6** fastest route (Dijkstra); **7** reachable locations (BFS); **8** best evacuation route from a location; **9** evacuation advice for all locations; **10** congestion report; **11** simulate a crowd; **12** congestion recommendations; **0** back. Alerts switch on at 80% of capacity and off at 70%. Zone hall occupancy comes from the seating data, so use options 3 and 20 in the main menu to change it.

5. Quick start (about 2 minutes)

Run `./a.out`, then enter: **12** (then **50**) → **18** → **8** → **20** (zone **0**) → **19** → **21** then **8** (location `Zone 0`) → **0** → **23** → **0**. Expected: 48 seated and 2 waitlisted (50 people, 48 seats), Zone 0 at 100% OVERCROWDED, an evacuation route to Exit West, and "Consistent". To replay a recorded session instead: `./a.out < samples/full_demo_input.txt`

6. Notes and troubleshooting

- "Grace period not over yet": use option 16 to advance the clock to at least 15 minutes, then option 17.
- "That seat is not free": the seat is taken, blocked or reserved; check option 8.
- The waiting list is not saved. After loading a file (option 14), run option 18.
- Option 22 replaces the default venue; restart the program to get it back. Name three halls exactly `Zone 0`, `Zone 1`, `Zone 2` to keep a custom venue linked to the seating data.
- If a graph prompt ends the program with "Input ended", the input ran out (for example a short input file).